SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO2 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	**Total Marks:**	1 * 100 = 100 (1 Question1)
CO2 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)*		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE AI / 2024	**Date:**	

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I **J KARTHIK / 192472136** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

1. **Chain-of-Thought Reasoning Task**

A financial application needs an LLM to analyze a multi-step calculation problem.

Task: Design an appropriate prompt that encourages reliable multi-step reasoning while returning only the required final structured result. Explain why a reasoning-oriented prompting strategy is appropriate.

1. Problem Statement

A financial institution wants to build an AI assistant that accurately solves financial calculations such as discounts, GST, loan EMI, interest calculations, and investment returns.

Since financial calculations involve multiple mathematical steps, incorrect reasoning may produce inaccurate results. Therefore, a reasoning-oriented prompting strategy is required to improve reliability while ensuring that the final output is returned in a structured format.

2. Analysis

Financial calculations usually require:

- Multiple intermediate calculations
- Correct mathematical operations
- Proper order of execution
- Accurate numerical values
- Consistent structured output

Examples include:

- Discount + GST Calculation
- EMI Calculation
- Compound Interest
- Tax Calculation
- Loan Eligibility

Without guided reasoning, an LLM may:

- Skip important calculation steps
- Miscalculate percentages
- Produce inconsistent outputs
- Mix explanations with final answers

Therefore, a structured reasoning strategy is appropriate.

3. Appropriate Prompt

You are an expert financial assistant.

Solve the given financial problem carefully.

Requirements:
1. Perform all calculations internally.
2. Verify every mathematical step.
3. Recheck the final answer before responding.
4. If an error is detected, correct it.
5. Return ONLY the following JSON format.

```
{
  "problem":"...",
  "final_amount":0,
  "currency":"INR",
  "status":"Verified"
}
```

Problem:
A laptop costs ₹50,000.
A discount of 10% is applied.
Then 18% GST is applied on the discounted price.
Find the final amount.

4. Structured Output Schema

JSON

```
{
  “problem”: “Laptop Price Calculation”,
  “final_amount”: 53100,
  “currency”: “INR”,
  “status”: “Verified”
}
```

5. Expected Output

JSON

```
{
  "problem": "Laptop Price Calculation",
  "final_amount": 53100,
  "currency": "INR",
  "status": "Verified"
}
```

6. Why a Reasoning-Oriented Prompt?

A reasoning-oriented prompt encourages the model to:

- Solve the problem systematically.
- Verify calculations before generating the answer.
- Reduce arithmetic mistakes.
- Produce reliable financial results.
- Return a predictable structured response.

This approach is particularly important for banking, accounting, taxation, insurance, and financial advisory systems where accuracy is essential.

Source Code

```python
import openai
from groq import Groq # Changed from OpenAI to Groq

client = Groq( # Changed from OpenAI to Groq
    api_key="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"
)

system_prompt = """
You are an expert financial assistant.

Solve every financial calculation carefully.

Perform internal reasoning.

Verify calculations before answering.

Return ONLY JSON.
"""

user_prompt = """
A laptop costs ₹50,000.

Discount = 10%

GST = 18%

Find the final amount.

Return JSON only.
"""

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile", # Changed model to a Groq model
    messages=[
        {"role":"system","content":system_prompt},
        {"role":"user","content":user_prompt}
    ],
    temperature=0
)

print(response.choices[0].message.content)
```

Output:

```
    ---------------------------------------- 143.7/143.7 kB 5.2 MB/s eta 0:00:00
```json
{
  "laptop_cost": 50000,
  "discount": 0.10,
  "discount_amount": 5000,
  "cost_after_discount": 45000,
  "gst": 0.18,
  "gst_amount": 8100,
  "final_amount": 53100
}
```
```

Test Cases:

```python
def calculate_final_price(price, discount, gst):
    discount_amount = price * discount / 100
    discounted_price = price - discount_amount
    gst_amount = discounted_price * gst / 100
    final_amount = discounted_price + gst_amount

    return {
        "original_price": price,
        "discount": discount,
        "gst": gst,
        "final_amount": round(final_amount, 2),
        "status": "Verified"
    }
```

```python
result = calculate_final_price(50000, 10, 18)

print(result)
```

```
{'original_price': 50000, 'discount': 10, 'gst': 18, 'final_amount': 53100.0, 'status': 'Verified'}
```

```python
result = calculate_final_price(20000, 20, 18)

print(result)
```

```
{'original_price': 20000, 'discount': 20, 'gst': 18, 'final_amount': 18880.0, 'status': 'Verified'}
```

```python
result = calculate_final_price(15000, 5, 12)

print(result)
```

```
{'original_price': 15000, 'discount': 5, 'gst': 12, 'final_amount': 15960.0, 'status': 'Verified'}
```

```python
result = calculate_final_price(80000, 15, 18)

print(result)
```

```
{'original_price': 80000, 'discount': 15, 'gst': 18, 'final_amount': 80240.0, 'status': 'Verified'}
```

2. Function Calling Schema

Function Name
calculate_final_price()

Function Schema
JSON

```
{
  "name": "calculate_final_price",
  "description": "Calculates final payable amount after applying discount and GST.",
  "parameters": {
    "type": "object",
    "properties": {
      "price": {
        "type": "number"
      },
      "discount_percentage": {
        "type": "number"
      },
      "gst_percentage": {
        "type": "number"
      }
    },
    "required": [
      "price",
      "discount_percentage",
      "gst_percentage"
    ]
  }
}
```

3. Structured Output (JSON Schema)

```
{
  "problem": "Laptop Price Calculation",
  "original_price": 50000,
  "discount_percentage": 10,
  "discount_amount": 5000,
  "discounted_price": 45000,
  "gst_percentage": 18,
  "gst_amount": 8100,
  "final_amount": 53100,
  "currency": "INR",
  "status": "Verified"
}
```

3. Test Cases

Test Case	Input	Expected Output	Result
TC1	₹50,000, 10%, 18% GST	₹53,100	Pass
TC2	₹20,000, 20%, 18% GST	₹18,880	Pass
TC3	₹15,000, 5%, 12% GST	₹15,960	Pass
TC4	₹80,000, 15%, 18% GST	₹80,240	Pass
TC5	Missing GST	Error Message	Pass

9. Advantages

- Improves calculation accuracy.
- Reduces mathematical errors.
- Produces consistent structured output.
- Easy to integrate with financial applications.
- Simplifies downstream processing of results.

10. Limitations

- Complex prompts may increase response time.
- The quality of the output depends on clear prompt design.
- Incorrect or ambiguous user input can still lead to inaccurate results.
- Requires validation for critical financial applications.

11. Conclusion

A reasoning-oriented prompting strategy is well suited for financial applications because it guides the model through a structured problem-solving process while ensuring that only the required structured result is returned. This improves reliability, consistency, and usability for real-world financial systems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____